

CPS 2231 Computer Programming

Chapter 3: Object-Oriented Programming Concepts

Dr. Hamza Djigal

Assistant Professor,

Department of Computer Science,
Wenzhou-Kean University

dhamza@kean.edu

Semester: Fall 2025

September 24, 2025
(Week 4)



温州肯恩大学
WENZHOU-KEAN UNIVERSITY

- 1 Objective
- 2 Definition
- 3 OOP Concepts in Java
- 4 Objects and Classes in Java
- 5 Methods in Java
- 6 Constructors in Java
- 7 static and this keywords
- 8 Practice

- 1 **Objective**
- 2 Definition
- 3 OOP Concepts in Java
- 4 Objects and Classes in Java
- 5 Methods in Java
- 6 Constructors in Java
- 7 static and this keywords
- 8 Practice

Objective

By the end of this lesson, you will be able to describe and use the following Java Object Oriented-Programming (OOP) Concepts:

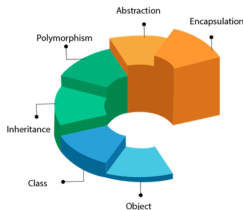
- Object and Class
- Constructors in Java
- Method in java
- Inheritance (association, aggregation, composition)
- Polymorphism
- Abstraction
- Encapsulation
- static and this keywords in Java

- 1 Objective
- 2 Definition**
- 3 OOP Concepts in Java
- 4 Objects and Classes in Java
- 5 Methods in Java
- 6 Constructors in Java
- 7 static and this keywords
- 8 Practice

Definition

Object-Oriented Programming (OOP)

- OOP refers to languages that use objects in programming, they use objects as a primary source to represent what is to happen in the code.
- OOP is a methodology to design a program using classes and objects.
- OOP simplifies software development and maintenance by the following concepts: Object; Class; Inheritance; Polymorphism Abstraction; and Encapsulation.



- 1 Objective
- 2 Definition
- 3 OOP Concepts in Java**
- 4 Objects and Classes in Java
- 5 Methods in Java
- 6 Constructors in Java
- 7 static and this keywords
- 8 Practice

Advantage of OOP

1. Without OOP (Procedural approach)

```
1 name1 = "Alice"  
2 age1 = 20  
3 gpa1 = 3.8  
4  
5 name2 = "Bob"  
6 age2 = 22  
7 gpa2 = 3.5  
8  
9 print(name1, age1, gpa1)  
10 print(name2, age2, gpa2)
```

Problem without OOP: For every new student, we repeat variables and print logic. **Messy and not scalable.**

Advantage of OOP

1. With OOP (Object Style)

```
1 Class Student:
2     Properties: name, age, gpa
3     Method: displayInfo()
4
5 Create student1 = Student("Alice", 20, 3.8)
6 Create student2 = Student("Bob", 22, 3.5)
7 student1.displayInfo()
8 student2.displayInfo()
```

Advantage with OOP:

- **Encapsulation:** Data and actions for “Student” are kept together.
- **Reusability:** You can easily create more students without rewriting code.
- **Clarity:** Code looks more like the real world (Student is a thing with info and behavior).

OOP Concepts in Java

Object

- Object in Java is any entity that has state and behavior. It can be physical or logical.
- An Object can be defined as an instance of a class.
- An object contains an address and takes up some space in memory.



OOP Concepts in Java

Class

- A class is a collection of objects. It is a logical entity. Class doesn't consume any space.
- Using classes, you can create multiple objects with the same behavior instead of writing their code multiple times.

Inheritance in Java

- Inheritance is one of the OOP concept in which one object acquires the properties and behaviors of the parent object.
- Inheritance is creating a parent-child relationship between two classes.
- Inheritance offers robust and natural mechanism for organizing and structure of any software.

OOP Concepts in Java

Polymorphism in Java

Polymorphism refers to one of the OOPs concepts in Java which is the ability of a variable, object or function to take on multiple forms.

Example: A person at the same time can have different characteristics (he can be a father, a husband, and an employee).

Abstraction in Java

Abstraction hiding internal details and showing functionality is known as abstraction.

Example: sending SMS or phone call: You don't know the internal processing about.

OOP Concepts in Java

Encapsulation in Java

- **Encapsulation** is one of the best Java OOPs concepts of Bidding (wrapping) the data and code.
- In encapsulation concept, the variables of a class are always hidden from other classes. It can only be accessed using the methods of their current class.

Association

Association: is a relationship between two objects. In an association, one object can be associated with one object or many objects. There can be four types of association:

- One to One
- One to Many
- Many to One
- Many to Many

OOP Concepts in Java

Aggregation

Aggregation: is a concept, where all objects have their separate lifecycle. However, there is ownership such that child object can't belong to another parent object.

Example: A car cannot move without a wheel. The wheel object can exist without the car object, which proves to be an aggregation relationship.

Composition

Composition: is a specialized form of Aggregation. It is also called death relationship. Child objects do not have their lifecycle so when parent object is deleted, all child objects will also be deleted automatically.

- 1 Objective
- 2 Definition
- 3 OOP Concepts in Java
- 4 Objects and Classes in Java**
- 5 Methods in Java
- 6 Constructors in Java
- 7 static and this keywords
- 8 Practice

Object

An **object** has three characteristics:

- **State:** represents the data (value) of an object.
- **Behavior:** represents the behavior (functionality) of an object such as deposit, withdraw, etc.
- **Identity:** An object identity is typically implemented via a unique ID. The value of the ID is not visible to the external user. However, it is used internally by the JVM to identify each object uniquely.

Example: Bank Account Object

```
1 Object: BankAccount
2   State: holderName = "Alice", balance = 500,
   accountType = "Savings"
3   Behavior: deposit(), withdraw(), checkBalance()
4   Identity: accountNumber = 123456
```


Class in Java

Definition of a Class

A **class** in Java is a *group of object* that defines:

- **Attributes (fields):** the data/properties of objects
- **Behaviors (methods):** the actions objects can perform

Definition of an Instance (Object)

An **instance** of a class is a *concrete object* created from the class blueprint.

Each instance has its own values for the attributes but shares the methods defined in the class.

Real-World Example: Car

- **Class:** Car (blueprint for all cars)
- **Attributes:** brand, year
- **Methods:** start(), stop()
- **Instances (Objects):**
 - `myCar = new Car(); // Toyota Corolla 2020`
 - `friendCar = new Car(); // Tesla Model 3 2023`

Understand the concept of Java Classes and Objects with an example.

Let's take an example of developing a pet management system, i.e. meant for dogs. We will need various information about the dogs like different breeds of the dogs, the age, size, etc.

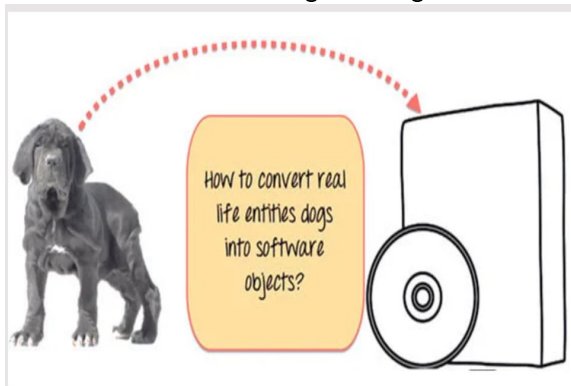


Image 1: Real Problem: DOG

Understand the concept of Java Classes and Objects with an example.

Let's take an example of developing a pet management system, i.e. meant for dogs. We will need various information about the dogs like different breeds of the dogs, the age, size, etc.



Image 2: Dog's Common Characteristics

Understand the concept of Java Classes and Objects with an example.

Let's take an example of developing a pet management system, i.e. meant for dogs. We will need various information about the dogs like different breeds of the dogs, the age, size, etc.

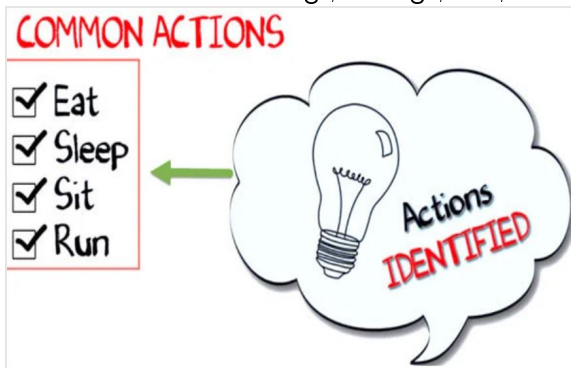


Image 3: Dog's Common Actioc

Understand the concept of Java Classes and Objects with an example.

Let's take an example of developing a pet management system, i.e. meant for dogs. We will need various information about the dogs like different breeds of the dogs, the age, size, etc.

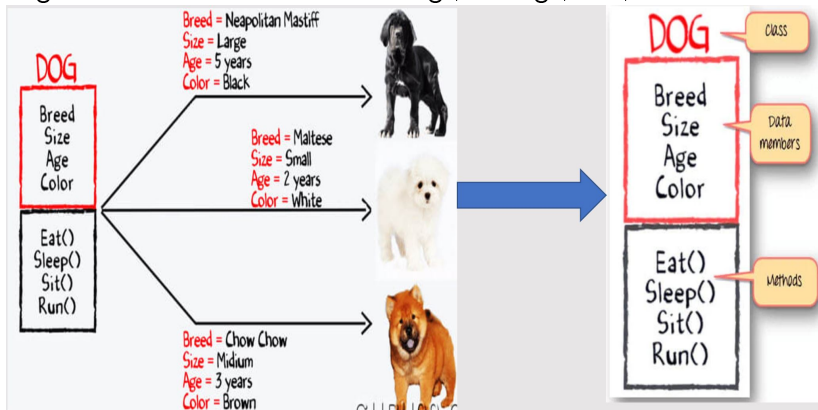


Image 3: Dog as Class

Java Class Example: Dog

```
1 public class Dog {
2     // Instance Variables
3     String breed;
4     String size;
5     int age;
6     String color;
7     // Method to get dog info
8     public String getInfo() {
9         return "Breed is: " + breed +
10             ", Size is: " + size +
11             ", Age is: " + age +
12             ", Color is: " + color;
13     }
14     public static void main(String[] args) {
15         Dog maltese = new Dog();
16         maltese.breed = "Maltese";
17         maltese.size = "Small";
18         maltese.age = 2;
19         maltese.color = "White";
20         System.out.println(maltese.getInfo());
21     }
22 }
```

Another Java Class and Instances Example

```
1 // Class definition
2 public class Car {
3     String brand;
4     int year;
5
6     public void start() {
7         System.out.println(brand + " starts!");
8     }
9     public static void main(String[] args) {
10        // Creating instances
11        Car myCar = new Car();
12        myCar.brand = "Toyota";
13        myCar.year = 2020;
14        myCar.start(); // Output: Toyota starts!
15
16        Car friendCar = new Car();
17        friendCar.brand = "Tesla";
18        friendCar.year = 2023;
19        friendCar.start(); // Output: Tesla starts!
20    }
21 }
```


Object's life cycle

```
class Person {}  
class ObjectLifeCycle {  
    Person person;  
}
```

← **Class Person**

← **Declaring a reference
variable of type Person**

No objects of class Person are created in the class `ObjectLifeCycle`; it declares only a variable of type Person.

```
class ObjectLifeCycle2 {  
    Person person = new Person();  
}
```

← **Declaring and initializing
a variable of type Person**

Object's life cycle

Object's life cycle

Difference in variable declaration and object creation

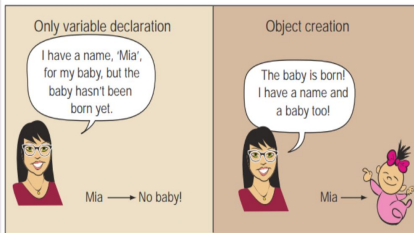


Figure 3.9 The difference between declaring a reference variable and initializing a reference variable

```
class ObjectLifeCycle2 {
    Person person = new Person();
    ObjectLifeCycle2() {
        new Person();
    }
}
```

← An unreferenced object

Variable

declaration & object creation

Object's life cycle

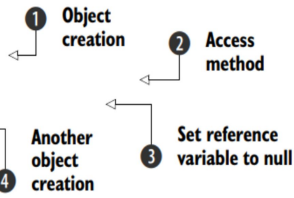
An object can become inaccessible if it goes out of scope:

```
class ObjectLifeCycle2 {
    Person person = new Person();
    ObjectLifeCycle2() {
        new Person();
    }
}
```

← An unreferenced object

An object of the class `Person` is created, but it can't be accessed using any reference variable. Creating an object in this manner will execute the relevant constructors of the class.

```
class ObjectLife1 {
    public static void main(String args[]) {
        Exam myExam = new Exam();
        myExam.setName("OCA Java Programmer 1");
        myExam = null;
        myExam = new Exam();
        myExam.setName("PHP");
    }
}
```



Variable Scope Analysis

```
1 public void myMethod() {  
2     int result = 88;  
3     if (result > 78) {  
4         Exam myExam1 = new Exam();  
5         myExam1.setName("Android");  
6     }  
7     else {  
8         Exam myExam2 = new Exam();  
9         myExam2.setName("MySQL");  
10    }  
11 }
```

Variable Scope Analysis

```
1 public void myMethod() {
2     int result = 88;
3     if (result > 78) {
4         Exam myExam1 = new Exam(); //Scope of local
5         variable myExam1
6         myExam1.setName("Android");
7     }
8     else {
9         Exam myExam2 = new Exam();
10        myExam2.setName("MySQL");
11    }
```

Definition

Garbage Collection (GC) is Java's automatic memory management process that:

- Identifies and removes objects no longer in use
- Reclaims memory occupied by unreachable objects
- Runs automatically in the background
- Prevents memory leaks and manual memory management errors

Rule of Garbage Collection

An object becomes eligible for garbage collection when it is no longer reachable

Common scenarios:

- Reference variable is set to `null`
- Reference is reassigned to another object
- Object goes out of scope
- Isolated circular references

Key Points to Remember

- GC is **automatic** - you don't control it directly
- `System.gc()` is only a **suggestion** to JVM
- Objects become eligible when **unreachable**
- Finalizer (`finalize()`) is **deprecated** (Java 9+)
- Different GC algorithms available (G1GC, Parallel, ZGC)

When GC Runs

- When JVM decides memory is low
- Periodically based on heap usage
- When explicitly suggested (`System.gc()`)
- Different for each GC algorithm

Note: GC behavior varies across JVM implementations

Garbage Collector

```
1 public class GarbageCollectionExample {
2
3     public static void main(String[] args) {
4         //Students are entering the classroom
5         String student1 = "Alice"; // Object 1
6         // created
7         String student2 = "Bob";    // Object 2
8         // created
9
10        System.out.println("Students in class: " + student1
11                             + ", " + student2);
12
13        //Alice is told to leave (object becomes
14        // unreachable)
15        student1 = null;
16
17        //Bob is replaced by Charlie
18        student2 = "Charlie"; // Now "Bob" has no
19        // variable pointing to it.
20
21        System.out.println("Now the student is: " +
22                             student2);
23    }
24 }
```

Garbage Collector

```
1 // The classroom is getting full. Let's suggest a
  cleanup.
2     // The Garbage Collector (janitor) may run
      now and remove "Alice" and "Bob".
3     System.gc(); // Just a suggestion, not a
      command!
4
5     System.out.println("The janitor (Garbage
      Collector) might have cleaned up by now
      .");
6 }
7 }
```

- 1 Objective
- 2 Definition
- 3 OOP Concepts in Java
- 4 Objects and Classes in Java
- 5 Methods in Java**
- 6 Constructors in Java
- 7 static and this keywords
- 8 Practice

Java Methods

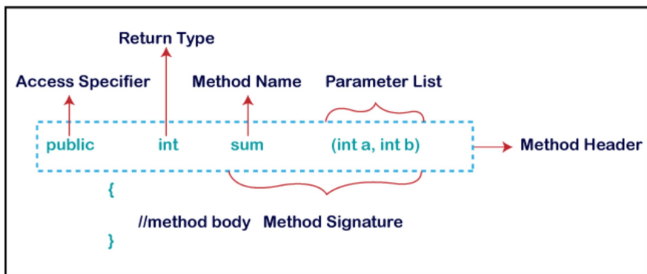
Definition

A **method** in Java is a block of code or a collection of statements grouped together to perform a certain task or operation. It provides **reusability of code**.

Key Point

Help to modify code using methods for cleaner and modular design.

Method Declaration



Java Method Components

Method Signature

Includes the **method name** and **parameter list**.

Access Specifier

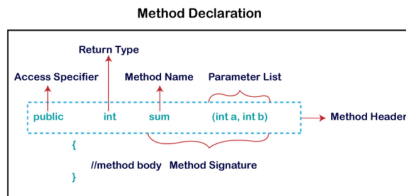
Access specifier (or modifier) specifies the visibility of the method. Java provides **four** types:

- **Public:** Accessible by all classes.
- **Private:** Accessible only in the class in which it is defined.
- **Protected:** Accessible within the same package or subclasses in a different package.
- **Default:** If no access specifier is used, the method is visible only within the same package.

Java Method Components

Other Method Components

- **Return Type:** The data type the method returns. Can be a primitive type, object, collection, or `void`.
- **Method Name:** Unique name corresponding to the functionality of the method.
- **Parameter List:** Comma-separated list of parameters enclosed in parentheses (empty if no parameters).
- **Method Body:** Contains all the actions to be performed, enclosed within curly braces.



Static Method in Java

Static Method

A method that has the `static` keyword is known as a **static method**.

- It can be called **without creating an object**.
- It can access and modify **static data members**.
- It is often invoked by using the **class name**.

```
1 class MyClass {  
2     static int count = 0;  
3     static void incrementCount() {  
4         count++;  
5         System.out.println("Count: " + count);  
6     }  
7  
8     public static void main(String[] args) {  
9         MyClass.incrementCount(); //called without  
10            object  
11         MyClass.incrementCount();  
12     } }
```


Instance Method in Java

Instance Method

An **instance method** is a method of the class that requires an **object** to be called or invoked. It cannot be called directly using the class name; you must create an instance of the class first.

Example of Instance Method

```
1 class MyClass {
2     int count = 0;
3     void incrementCount() {
4         count++;
5         System.out.println("Count: " + count);
6     }
7     public static void main(String[] args) {
8         MyClass obj = new MyClass(); //create object
9         obj.incrementCount(); //call instance method
10        obj.incrementCount();
11    }
12 }
```

Accessor Method in Java

Accessor Method (Getter)

An **accessor method** is a type of instance method that **reads the instance variable(s)** of a class.

- Usually prefixed with `get`, hence also called **getters**.
- Retrieves private variables or constants to access them outside the class scope.
- The return type of the getter matches the type of the variable it returns.

Accessor Method in Java

Example

```
1 class Person {  
2     private String name;  
3  
4     // Accessor method  
5     public String getName() {  
6         return name;  
7     }  
8  
9     public static void main(String[] args) {  
10         Person p = new Person();  
11         System.out.println(p.getName()); //call  
12             getter  
13     }
```

Mutator Method in Java

Mutator Method (Setter)

A **mutator method** is a type of instance method that **reads and modifies instance variable(s)** of a class.

- Usually prefixed with `set`, hence also called **setters**.
- Used to update a variable's value outside the class scope.
- Does **not return anything** (return type is `void`).
- Used to set the value of private fields safely.

Mutator Method in Java

Example

```
1 class Person {
2     private String name;
3
4     // Mutator method
5     public void setName(String n) {
6         name = n;
7     }
8
9     public static void main(String[] args) {
10         Person p = new Person();
11         p.setName("Alice");    // call setter
12     }
13 }
```

Why Accessors and Mutators?

Purpose of Getters and Setters

We need **accessors (getters)** and **mutators (setters)** to protect sensitive information in a class.

- Encapsulates private data and controls how it is accessed or modified.
- Protects information from illegal or unintended use.
- Provides a standard, maintainable interface for reading and updating private fields.

Abstract Method: Definition

Definition

- An **abstract method** does not have a method body (i.e., no implementation).
- It is always declared inside an **abstract class**.
- To create an abstract method, we use the `abstract` keyword.

```
1 //Syntax of Abstract Method
2 abstract class ClassName {
3     // Abstract method declaration
4     abstract returnType methodName(parameters);
5
6     // Example: abstract void makeSound();
7
8     // You can also have regular methods
9     void regularMethod() {
10         // method body
11     }
12 }
```

Example of Abstract Method

```
1 // Abstract class
2 abstract class Demo {
3     // Abstract method declaration
4     abstract void display();
5 }
6
7 // Concrete subclass implementing abstract method
8 public class MyClass extends Demo {
9     // Method implementation
10    void display() {
11        System.out.println("Abstract method?");
12    }
13
14    public static void main(String args[]) {
15        // Creating object of abstract class
16        // through subclass
17        Demo obj = new MyClass();
18        // Invoking abstract method
19        obj.display();
20    }
```


- 1 Objective
- 2 Definition
- 3 OOP Concepts in Java
- 4 Objects and Classes in Java
- 5 Methods in Java
- 6 Constructors in Java**
- 7 static and this keywords
- 8 Practice

Constructors in Java

Definition

- In Java, a **constructor** is a block of code similar to a method.
- It is called when an instance of the class is created. Memory for the object is allocated at this time.
- It is a special type of method used to initialize the object.

Key Points

- Every time an object is created using the `new` keyword, at least one constructor is called.
- If no constructor is defined, Java provides a default constructor automatically.

Rules for Creating a Constructor

- ① Constructor name must be the same as the class name.
- ② A constructor must have **no explicit return type**.

Constructors in Java

Types of Constructors

- **Default Constructor (No-arg constructor)** – takes no arguments.
- **Parameterized Constructor** – takes arguments to initialize an object with specific values.

Default Constructor in Java

Default Constructor

- If there is no constructor in a class, compiler automatically creates a default constructor.
- **Purpose:** The default constructor is used to provide the default values to the object like 0, null, etc., depending on the type.

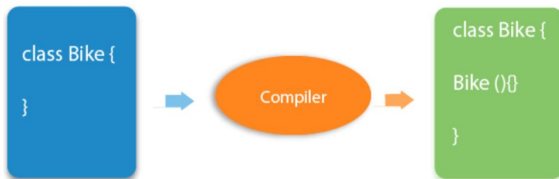


Fig. 1: Compiler create default constructor

Default Constructor Example

Default Constructor Example

```
1 class Demo {
2     int value1; int value2;
3     // Default constructor
4     Demo() {
5         value1 = 10;
6         value2 = 20;
7         System.out.println("This is inside the
8             constructor");
9     }
10    // Method to display values
11    public void display() {
12        System.out.println("Value1 = " + value1);
13        System.out.println("Value2 = " + value2);
14    }
15
16    public static void main(String[] args) {
17        Demo d1 = new Demo(); // Calls constructor
18        d1.display();          // Display values
19    }
```

Parameterized Constructor Example

Parameterized Constructor Example

```
1 // Parameterized constructor example
2 class Student {
3     String name;
4
5     // Parameterized constructor
6     Student(String n) {
7         name = n;
8     }
9
10    void display() {
11        System.out.println("Name: " + name);
12    }
13
14    public static void main(String[] args) {
15        Student s2 = new Student("Alice"); //
16        s2.display();                       // calls parameterized constructor
17        // Output: Name: Alice
18    }
```

Constructor Overloading in Java

Definition

Constructor overloading in Java is a technique of **having more than one constructor with different parameter lists**.

Example

```
1 class Student {
2     String name; int age;
3     // Constructor 1: no parameters
4     Student() {
5         name = "Unknown";
6         age = 0;
7     }
8     // Constructor 2: one parameter
9     Student(String n) {
10        name = n;    age = 0;
11    }
12    // Constructor 3: two parameters
13    Student(String n, int a) {
14        name = n;    age = a;
15    }
```

Constructor Overloading in Java

Example

```
1  class Student {
2      String name;  int age;
3      // Constructor 1: no parameters
4      Student() {
5          name = "Unknown";          age = 0;
6      }
7      // Constructor 2: one parameter
8      Student(String n) {
9          name = n;          age = 0;
10     }
11     // Constructor 3: two parameters
12     Student(String n, int a) {
13         name = n; age = a;
14     }
15     void display() {
16         System.out.println("Name: " + name + ", Age
17                               : " + age);
18     }
19 }
```


Constructor Overloading in Java

Example

```
1  class Student {  
2      .....  
3      .....  
4  
5  
6      .....  
7  
8      public static void main(String[] args) {  
9          Student s1 = new Student();           //  
10             Constructor 1  
11          Student s2 = new Student("Alice");    //  
12             Constructor 2  
13          Student s3 = new Student("Bob", 21);  //  
14             Constructor 3  
15  
16          s1.display(); // Name: Unknown, Age: 0  
17          s2.display(); // Name: Alice, Age: 0  
18          s3.display(); // Name: Bob, Age: 21  
19      }  
20  }
```

Difference Between Constructor and Method in Java

Java Constructor	Java Method
Used to initialize the state of an object.	Used to expose or define the behavior of an object.
Must not have a return type.	Must have a return type.
Invoked implicitly.	Invoked explicitly.
Compiler provides a default constructor if none is defined.	Compiler never provides a default method.
Name must be the same as the class name.	Name may or may not be the same as the class name.

Java Constructors: Access Levels and Usage

1. All Access Levels: You can use public, protected, default, and private constructors.

```
1 class Demo {  
2     public Demo() { }           // public  
3     protected Demo(int x) { } // protected  
4     Demo(String s) { }         // default  
5     private Demo(double d) { } // private  
6 }
```

2. Called on Object Creation: A constructor runs automatically when you create an object.

```
1 class Test {  
2     Test() {  
3         System.out.println("Constructor called!");  
4     }  
5     public static void main(String[] args) {  
6         Test t = new Test(); // prints immediately  
7     }  
8 }
```

Java Constructors: Access Levels and Usage

3. Assign Default Values: You can set default values inside the constructor.

```
1 class Student {  
2     String name; int age;  
3     Student() {  
4         name = "Default Name";  
5         age = 18;  
6     }  
7 }
```

4. Pass Parameters: You can also pass values into a constructor.

```
1 class Person {  
2     String name; int age;  
3     Person(String n, int a) {  
4         name = n; age = a;  
5     }  
6     public static void main(String[] args) {  
7         Person p = new Person("Ali", 25);  
8     }  
9 }
```

- 1 Objective
- 2 Definition
- 3 OOP Concepts in Java
- 4 Objects and Classes in Java
- 5 Methods in Java
- 6 Constructors in Java
- 7 static and this keywords**
- 8 Practice

Understanding the static Keyword in Java

Definition and Usage

- The **static** keyword is used for memory management.
- Makes a member belong to the **class**, not objects.
- Can be applied to:
 - **Static Variables**: one copy shared by all objects.
 - **Static Methods**: can be called without creating an object.
 - **Static Blocks**: executed once when class is loaded.
 - **Static Nested Classes**: independent of outer class object.

Why is the main() Method Static?

Reason

- JVM needs to call **main** before any object is created.
- If main were non-static, JVM would first create an object:

```
1 public class Test {  
2     public void main(String[] args) {  
3         System.out.println("This won't work!");  
4     }  
5 }
```

- This would cause **extra memory allocation**.
- Static main allows JVM to start the program efficiently.

Can We Execute a Program Without main()?

Before JDK 1.7

```
1 class Test {  
2     static {  
3         System.out.println("Hello from static block  
4             !");  
5         System.exit(0); // program exits here  
6     }  
}
```

- Static block executed automatically.
- Program could run even without main().

Since JDK 1.7

- JVM requires **main()** as the entry point.
- Static blocks alone will **not work**.

Understanding the **this** Keyword in Java

Definition: `this` is a reference variable in Java that refers to the **current object**.

Usage of Java `this` Keyword

- **Refer current class instance variable:** Resolves ambiguity between instance variables and parameters.
- **Invoke current class method:** Can call another method of the same object (implicitly or explicitly).
- **Invoke current class constructor:** Use `this()` for constructor chaining.
- **Pass current object as argument:** Send the current object to another method.
- **Pass current object in constructor call:** Send the object to another class' s constructor.
- **Return current object:** Useful for method chaining (fluent API design).

Understanding the **this** Keyword in Java

Without this (Problem)

```
1 class Student {
2     String name; // instance variable
3     Student(String name) {
4         name = name; // ambiguous, assigns parameter to itself
5     }
6     void display() {
7         System.out.println(name); // prints null
8     }
9 }
```

With this (Solution)

```
1 class Student {
2     String name;
3     Student(String name) {
4         this.name = name; // assigns parameter to instance variable
5     }
6     void display() {
7         System.out.println(name); // prints actual name
8     }
9 }
```

Examples of this Keyword Usage in Java

- Refer current instance variable:

```
1 class Student {  
2     String name;  
3     Student(String name) {  
4         this.name = name; //distinguishes instance variable  
5     }  
6 }
```

- Invoke current class method:

```
1 class Demo {  
2     void hello() {System.out.println("Hello"); }  
3     void greet() {this.hello(); } //calls the method hello()  
4     }  
5 }
```

- Invoke current class constructor:

```
1 class Box {  
2     int w,h;  
3     Box() { this(10,20); } //calls parameterized constructor  
4     Box(int w,int h) { this.w=w; this.h=h; }  
5 }
```

Examples of this Keyword Usage in Java

- Pass current object as method argument:

```
1 class Printer {  
2     void print(Student s){ System.out.println(s.name); }  
3 }  
4 Student s = new Student("Alice");  
5 new Printer().print(s); //or print(this) inside Student  
    method
```

- Pass current object in constructor call:

```
1 class Manager {  
2     Manager(Employee e) {System.out.println(e.name); }  
3 }  
4 class Employee {  
5     String name="Bob";  
6     Employee() { new Manager(this); }  
7 }
```

- Return current object (method chaining):

```
1 class Student {  
2     String name;  
3     Student setName(String n){ this.name=n; return this; }  
4 }  
5 Student s = new Student().setName("Alice");
```

- 1 Objective
- 2 Definition
- 3 OOP Concepts in Java
- 4 Objects and Classes in Java
- 5 Methods in Java
- 6 Constructors in Java
- 7 static and this keywords
- 8 Practice**

Practice 1: Building Volume

Problem

Create a class `Building` with:

- **Instance variables:** height, width, breadth
- **Method:** `volume()` to calculate the volume

Create another class `BuildingDemo` which:

- Creates instances of `Building`
- Displays the volume of each building

Practice 2: Compute $\sin x$

Problem

Write a Java program to compute $\sin x$ without using `Math.sin()`. Hints:

- Use Taylor series: $\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots$
- Implement it inside a class with a method (e.g. `calculateSin(double x)`)
- Create a driver class to take input and display the result

Practice 3: Student Marks

Problem

Create a class `Student` with:

- **Instance variables:** `name`, `rollNumber`, `marks` (array)
- **Constructor:** initializes all values
- **Methods:**
 - `average()` returns average marks
 - `display()` prints name, roll number and average

Create a `StudentDemo` class to:

- Create at least two `Student` objects
- Call `display()` for each

Practice 4: Bank Account & Customer

Problem

Create a class `BankAccount`:

- **Variables:** `accountNumber`, `accountHolderName`, `balance`
- **Constructor:** initializes all fields
- **Methods:** `deposit()`, `withdraw()`, `display()`

Create another class `Customer`:

- **Variables:** `customerName` and one `BankAccount` object
- **Constructor:** initializes the customer name and account
- **Method:** `displayCustomerInfo()` shows the customer's name and account details

Create a `BankDemo` class to:

- Create at least one `Customer` object with a `BankAccount`
- Perform deposit/withdraw operations
- Display all details using the methods